Universidade Católica de Brasília

Disciplina: Análise de Algoritmos

Alunos: Carlos Eduardo Lima de Oliveira

Henrique Kümmel do Rosário

Aplicação do Teorema Mestre

Análise teórica e experimental de três algoritmos recursivos

Algoritmo	Recorrência de tempo	Classificação
Máximo por divisão em quatro partes	$4T(n/4) + \Theta(1)$	Caso 1 · $\Theta(n)$
Busca binária recursiva	$T(n/2) + \Theta(1)$	Caso 2 · $\Theta(\log n)$
Soma por recursão e varredura	$T(n/2) + \Theta(n)$	Caso 3 · $\Theta(n)$

Resumo

Este relatório apresenta a modelagem, a implementação em C e a avaliação experimental de três algoritmos enquadrados nos casos clássicos do Teorema Mestre. As complexidades são demonstradas por indução sobre contagens exatas de operações. A avaliação contempla 24 tamanhos de entrada, sete amostras temporais por tamanho e instrumentação independente. As contagens confirmam as expressões deduzidas, enquanto os tempos apresentam crescimento compatível com as classes assintóticas, consideradas as limitações do ambiente.

Palavras-chave: análise de algoritmos; recorrências; Teorema Mestre; indução matemática; avaliação experimental.

Adequação às restrições

Os Casos 1 e 2 utilizam, respectivamente, $a = 4$ e $a = 1$, ambos diferentes de 2. No Caso 2, $a = 1$ e $b = 2$, satisfazendo também a condição $a \neq b$.

29 de outubro de 2026

Medições realizadas em ambiente Linux remoto compartilhado. Código-fonte e dados experimentais disponibilizados para reprodução.

Repositório do trabalho

https://github.com/kaduolliver/Teorema-Mestre

FUNDAMENTAÇÃO

1. Introdução

A análise de algoritmos recursivos depende da identificação do número de subproblemas, do fator de redução das entradas e do custo de processamento realizado em cada chamada. O Teorema Mestre fornece resultados assintóticos para uma classe de recorrências associadas a esse tipo de decomposição.

O objetivo geral consiste em aplicar os três casos do teorema, confirmar os resultados por demonstrações matemáticas e comparar as previsões com dados obtidos por implementações em C. São analisados três problemas distintos: determinação do maior elemento de um vetor, busca em vetor ordenado e soma de elementos.

Os objetivos específicos abrangem a dedução das recorrências, a identificação de seus parâmetros, o estabelecimento de limites assintóticos superiores e inferiores e a avaliação experimental do crescimento do custo computacional.

2. Revisão teórica

$$T(n) = aT(n/b) + f(n), \text{ com } a \geq 1,\ b > 1 \text{ e } T(1) = \Theta(1).$$

Símbolo	Definição
n	Número de elementos do subproblema corrente.
T(n)	Custo total de execução para uma entrada de tamanho n.
a	Número de chamadas recursivas executadas em cada etapa não terminal.
b	Fator constante de redução do tamanho de cada subproblema.
f(n)	Custo local, excluído o processamento das chamadas recursivas.

A análise adota o modelo RAM de custo unitário para operações sobre palavras de máquina. As provas consideram potências exatas de b, eliminando arredondamentos. As implementações possuem limites de entrada compatíveis com os tipos inteiros utilizados.

Termo de referência

$$p = \log_b a;\ \text{o enquadramento compara } f(n) \text{ com } n^p.$$

A notação $\Theta(g(n))$ exige constantes positivas que limitem o custo superior e inferiormente por múltiplos de g(n), para entradas suficientemente grandes. A base constante do logaritmo não altera a classe assintótica.

FUNDAMENTAÇÃO

2. Revisão teórica · casos clássicos

Para a recorrência apresentada e custo local assintoticamente não negativo, a versão clássica do Teorema Mestre estabelece os seguintes resultados [1].

Caso	Hipóteses	Resultado
1	$f(n) = O(n^{p-\varepsilon})$, para algum $\varepsilon > 0$	$T(n) = \Theta(n^p)$
2	$f(n) = \Theta(n^p)$	$T(n) = \Theta(n^p \log n)$
3	$f(n) = \Omega(n^{p+\varepsilon})$, $\varepsilon > 0$; $a\, f(n/b) \leq c\, f(n)$, com $0 < c < 1$	$T(n) = \Theta(f(n))$

2.1 Interpretação dos casos

No Caso 1, f(n) é polinomialmente menor que o termo de referência; predomina o custo associado aos níveis profundos da árvore de recursão. No Caso 2, o trabalho por nível mantém a mesma ordem e a profundidade acrescenta um fator logarítmico. No Caso 3, o custo local dos níveis superiores determina a ordem total, desde que satisfeita a regularidade.

As hipóteses dos Casos 1 e 3 exigem uma diferença polinomial, expressa pelo fator n^{ε}. Uma relação de crescimento estritamente menor ou maior, isoladamente, não é suficiente para aplicar esses casos.

2.2 Critério de modelagem

O parâmetro a corresponde ao número de chamadas executadas em uma etapa, e não ao número de alternativas presentes no código. Chamadas localizadas em ramos mutuamente exclusivos não são somadas. Analogamente, uma partição dos dados pode ser processada por um laço sem gerar uma chamada recursiva adicional.

2.3 Contagem exata e custo total

Define-se Q(n) como uma contagem de operações representativas: visitas a chamadas nos Problemas 1 e 2 e leituras de elementos no Problema 3. Q(n) não representa tempo decorrido nem o total de instruções de máquina.

As demonstrações resolvem as recorrências exatas de Q(n) por indução em k, para $n = b^k$. A relação $T(n) = \Theta(Q(n))$ é justificada pelo custo constante das operações contadas e pelos limites do trabalho de controle adicional. Essa abordagem estabelece a correspondência entre a contagem instrumentada e a análise assintótica.

Resultados a demonstrar:
$Q_1(n) = (4n - 1)/3$;
$Q_2(n) = \log_2 n + 1$;
$Q_3(n) = n$.

PROCEDIMENTO

3. Metodologia

3.1 Objetivos específicos e seleção dos algoritmos

Os algoritmos selecionados isolam três relações distintas entre a expansão recursiva e o custo local. A busca pelo máximo utiliza quatro chamadas com combinação constante; a busca binária utiliza uma chamada com custo local constante; a soma utiliza uma chamada e uma varredura linear.

Problema	Parâmetros estruturais	Tamanhos avaliados
1 · Máximo	a = 4; b = 4; combinação Θ(1).	4^3, 4^4, ..., 4^{10}
2 · Busca	a = 1; b = 2; decisão Θ(1).	2^4, 2^7, ..., 2^{25}
3 · Soma	a = 1; b = 2; varredura Θ(n).	2^6, 2^8, ..., 2^{20}

3.2 Ferramentas e ambiente experimental

Componente	Especificação
Linguagem e compilador	C11; GCC 13.3.0; -O2 -Wall -Wextra.
Controle da recursão	-fno-optimize-sibling-calls; atributos noinline e noipa.
Temporização	clock_gettime(CLOCK_MONOTONIC) [2].
Instrumentação	Executável independente compilado com -DINSTRUMENT.
Tratamento e gráficos	Python 3.12.14, NumPy, pandas e Matplotlib.
Hardware e sistema	AMD EPYC 9V74; Linux 6.18.55 x86_64; glibc 2.39.

As execuções ocorreram sequencialmente, com afinidade no processador lógico 0, em ambiente Linux remoto compartilhado. A resolução nominal do relógio é 1 ns; esse valor não estabelece a precisão efetiva dos intervalos medidos.

3.3 Entradas e verificação funcional

Cada vetor contém a sequência 0, 1, ..., n−1. As respostas de referência são n−1 para o máximo, 0 para a busca pelo alvo ausente n e n(n−1)/2 para a soma. A ordenação prévia é uma precondição da busca e não integra seu custo.

A verificação contemplou todos os resultados registrados, 15 casos pequenos com AddressSanitizer e UBSan e testes adicionais com valores variados e alvos presentes, ausentes e repetidos. A detecção de vazamentos foi desativada por restrição do ambiente. Os limites dos argumentos mantêm os dados e resultados compatíveis com uint32_t e uint64_t.

PROCEDIMENTO

3. Metodologia · coleta e análise

3.4 Protocolo de medição

Foram avaliados oito tamanhos por algoritmo, totalizando 24 configurações. A ordem foi embaralhada com semente 20261029. Para cada configuração, uma execução verifica o resultado e a calibração aumenta o lote até atingir 5 ms ou o limite de 1.048.576 chamadas. O lote é então ajustado para uma duração estimada de 20 ms, respeitando esse limite.

Cada configuração possui sete amostras temporais. Cada amostra mede o lote com duas leituras do relógio e divide a duração pelo número de chamadas. Alocação, preenchimento, validação e gravação são externos ao intervalo; o laço do coletor e o despacho das chamadas integram a medição.

A campanha produziu 192 registros em um CSV único: 168 amostras temporais e 24 contagens instrumentadas. Cada linha temporal contém uma média de lote. As contagens são obtidas em executável separado para evitar que a atualização dos contadores afete o tempo.

3.5 Tratamento estatístico e curvas de referência

Para cada configuração, calculam-se a mediana e os quartis Q1 e Q3 das sete médias de lote, com interpolação linear dos percentis. O intervalo Q1-Q3 caracteriza dispersão entre amostras e não constitui intervalo de confiança. Os gráficos e tabelas usam microssegundos (1 µs = 10^{-6} s).

$g_1(n) = n$; $g_2(n) = 1 + \log_2 n$; $g_3(n) = n$.
$\hat{t}(n) = c\, g(n)$, sendo c a mediana de mediana$(n)/g(n)$
nos três maiores tamanhos de cada algoritmo.

O termo unitário de g_2 representa a chamada-base e preserva a classe $\Theta(\log n)$ para $n \geq 2$. A calibração de c utiliza os próprios dados e permite comparar formas de crescimento; não fornece previsão independente de tempo absoluto.

3.6 Adequação dos métodos e limitações

O código em C e a instrumentação explícita relacionam o programa às recorrências. A medição em lotes reduz a contribuição relativa do relógio, especialmente na busca. A afinidade reduz migrações entre processadores, mas não isola preempção, frequência de CPU ou competição por recursos.

A campanha usa uma entrada determinística por tamanho e a repete em cada lote, favorecendo caches e predição de desvios. Os atributos e opções de compilação preservam a estrutura recursiva e afetam constantes. As observações são restritas a esse ambiente e a intervalos finitos. A complexidade é estabelecida pelas provas; os tempos avaliam sua compatibilidade empírica.

ANÁLISE TEÓRICA

4.1 Problema 1 · determinação do máximo

4.1.1 Definição e decomposição

Dado um vetor v de $n = 4^k$ elementos, determina-se max(v). O vetor é particionado em quatro segmentos de mesmo tamanho. Cada segmento é processado recursivamente e os quatro máximos parciais são combinados por três comparações. O caso-base retorna o único elemento.

4.1.2 Implementação recursiva

```
REC uint32_t maior4(const uint32_t *v, size_t n) {
    CONTA();
    if (n == 1) return v[0];
    size_t m = n / 4;
    uint32_t a = maior4(v, m), b = maior4(v + m, m);
    uint32_t c = maior4(v + 2*m, m), d = maior4(v + 3*m, m);
    uint32_t maior = a > b ? a : b;
    if (c > maior) maior = c;
    if (d > maior) maior = d;
    return maior;
}
```

Os deslocamentos de ponteiro delimitam segmentos sem copiar elementos. REC preserva as chamadas no experimento; CONTA() é ativada exclusivamente na versão instrumentada. As definições e o coletor completo constam no Apêndice A.

4.1.3 Correção e modelo de custo

No caso-base, o retorno coincide com o máximo do segmento unitário. Admitindo a correção dos quatro resultados recursivos, o maior entre eles é o máximo da união dos segmentos. As partições são disjuntas e cobrem o vetor, o que estabelece a correção da decomposição.

Componente	Caracterização
Chamadas executadas	Quatro chamadas sobre n/4 elementos: a = 4 e b = 4.
Custo local	Três comparações e controle constante: f(n) = Θ(1).
Contagem $Q_1(n)$	Uma unidade por chamada, incluindo as folhas.

$$T_1(n) = 4T_1(n/4) + \Theta(1);\ T_1(1) = \Theta(1).$$

Exemplo de execução: para [8, 3, 10, 4], os resultados unitários são combinados e o retorno é 10. Para n = 16, a árvore possui 1 + 4 + 16 = 21 chamadas.

ANÁLISE TEÓRICA

4.1 Problema 1 · enquadramento e prova

4.1.4 Aplicação do Caso 1

$$a = 4;\ b = 4;\ p = \log_4 4 = 1.$$
$$n^{\log_4 4} = n;\ f(n) = \Theta(1).$$

Tomando $\varepsilon = 1$, obtém-se $n^{p-\varepsilon} = 1$ e $f(n) = O(n^{p-\varepsilon})$. Logo, o custo local é polinomialmente menor que o termo de referência e a recorrência satisfaz o Caso 1.

$$T_1(n) = \Theta(n^{\log_4 4}) = \Theta(n).$$

A condição $a \neq 2$ é satisfeita por $a = 4$. O número de folhas é n e a profundidade é $\log_4 n$. O custo de combinação permanece constante por chamada, enquanto o número de subproblemas cresce a cada nível.

4.1.5 Demonstração por indução

Para a contagem de visitas, $Q_1(1) = 1$ e $Q_1(n) = 4Q_1(n/4) + 1$. Afirma-se que $Q_1(n) = (4n - 1)/3$ para $n = 4^k$, k inteiro não negativo.

Base: $k = 0$ implica $n = 1$. A expressão fornece $(4 \cdot 1 - 1)/3 = 1$, em concordância com o caso-base.

Hipótese de indução: para o tamanho $n/4$, $Q_1(n/4) = [4(n/4) - 1]/3 = (n - 1)/3$.

Passo indutivo: substituindo a hipótese na recorrência,

$$Q_1(n) = 4Q_1(n/4) + 1$$
$$= 4(n - 1)/3 + 1 = (4n - 1)/3.$$

A expressão vale, portanto, para todas as potências de 4. Para $n \geq 1$, $n \leq Q_1(n) \leq 4n/3$, de modo que $Q_1(n) = \Theta(n)$. Cada visita tem trabalho constante positivo e limitado, estabelecendo $T_1(n) = \Theta(Q_1(n)) = \Theta(n)$.

Q_1 contabiliza chamadas, não comparações entre elementos. O número destas comparações é $n-1$. A memória auxiliar da implementação é $\Theta(\log n)$, correspondente à pilha de chamadas; o armazenamento do vetor de entrada é $\Theta(n)$.

ANÁLISE TEÓRICA

4.2 Problema 2 · busca binária

4.2.1 Definição e variante adotada

Dado um vetor ordenado de $n = 2^k$ elementos e um alvo x, decide-se se x pertence ao vetor. A variante adotada compara x com o primeiro elemento da metade direita e recorre em apenas uma das metades. A verificação de igualdade ocorre no segmento unitário.

4.2.2 Implementação recursiva

```
REC int busca(const uint32_t *v, size_t n, uint32_t alvo) {
    CONTA();
    if (n == 1) return v[0] == alvo;
    size_t m = n / 2;
    if (alvo < v[m]) return busca(v, m, alvo);
    return busca(v + m, m, alvo);
}
```

A metade direita inclui v[m], assegurando que ambas as alternativas tenham exatamente n/2 elementos. O retorno antecipado da primeira alternativa torna os caminhos mutuamente exclusivos. Assim, a = 1 e b = 2, embora existam duas linhas recursivas no código.

4.2.3 Correção e exemplo de execução

Se x < v[m], nenhum elemento à direita pode ser igual a x. Se x ≥ v[m], a metade direita contém a posição de x quando ele existe; no caso de igualdade, v[m] permanece nessa metade. A ordenação mantém a propriedade a cada etapa e o caso-base decide a pertinência.

Segmento corrente	Condição para x = 6	Segmento selecionado
[0, 1, 2, 3, 4, 5, 6, 7]	6 ≥ 4	[4, 5, 6, 7]
[4, 5, 6, 7]	6 ≥ 6	[6, 7]
[6, 7]	6 < 7	[6]
[6]	6 = 6	Retorno 1

$$T_2(n) = T_2(n/2) + \Theta(1);\ T_2(1) = \Theta(1).$$

Para n = 8, o caminho contém quatro chamadas. Nesta variante, alvos presentes e ausentes percorrem o mesmo número de níveis para potências de 2. O custo de ordenar uma entrada arbitrária não integra a operação de busca analisada.

ANÁLISE TEÓRICA

4.2 Problema 2 · enquadramento e prova

4.2.4 Aplicação do Caso 2

$a = 1$; $b = 2$; $p = \log_2 1 = 0$.
$n^{\log_2 1} = n^0 = 1$; $f(n) = \Theta(1)$.

O custo local possui a mesma ordem do termo de referência: $f(n) = \Theta(n^p)$. A recorrência satisfaz, portanto, o Caso 2. Cada nível executa uma chamada com custo constante, e a profundidade é $\log_2 n$.

$$T_2(n) = \Theta(n^p \log n) = \Theta(\log n).$$

O teorema admite $a \geq 1$, incluindo $a = 1$. As restrições do enunciado são atendidas: $a = 1 \neq 2$ e $a = 1 \neq b = 2$.

4.2.5 Demonstração por indução

Seja $Q_2(n)$ o número de chamadas. $Q_2(1) = 1$ e $Q_2(n) = Q_2(n/2) + 1$. Afirma-se que $Q_2(n) = \log_2 n + 1$ para $n = 2^k$, k inteiro não negativo.

Base: $k = 0$ implica $n = 1$, para o qual $\log_2 1 + 1 = 1$. A chamada terminal integra a contagem.

Hipótese de indução: para $n/2$, $Q_2(n/2) = \log_2(n/2) + 1$.

Passo indutivo: aplicando $\log_2(n/2) = \log_2 n - 1$,

$$\begin{aligned} Q_2(n) &= Q_2(n/2) + 1 \\ &= \log_2(n/2) + 2 \\ &= \log_2 n - 1 + 2 = \log_2 n + 1. \end{aligned}$$

A expressão é válida para todas as potências de 2. Para $n \geq 2$, $\log_2 n \leq Q_2(n) \leq 2 \log_2 n$. Logo $Q_2(n) = \Theta(\log n)$. Como cada chamada possui custo constante positivo, $T_2(n) = \Theta(Q_2(n)) = \Theta(\log n)$.

O caso $n = 1$ é tratado separadamente e não altera a classificação assintótica. A memória auxiliar desta versão recursiva é $\Theta(\log n)$. A opção -fno-optimize-sibling-calls preserva as chamadas na compilação usada na temporização.

ANÁLISE TEÓRICA

4.3 Problema 3 · soma de elementos

4.3.1 Definição e decomposição

Dado um vetor v de $n = 2^k$ elementos, determina-se $\sum v[i]$. A primeira metade é processada recursivamente, enquanto a segunda é percorrida por um laço. O resultado combina as duas somas. Para n = 1, retorna-se o elemento.

4.3.2 Implementação recursiva

```
REC uint64_t soma(const uint32_t *v, size_t n) {
    if (n == 1) { CONTA(); return v[0]; }
    size_t m = n / 2;
    uint64_t total = soma(v, m);
    for (size_t i = m; i < n; ++i) { total += v[i]; CONTA(); }
    return total;
}
```

4.3.3 Correção e modelo de custo

As duas metades são disjuntas e cobrem o vetor. Admitindo que a chamada retorne a soma da primeira metade, o laço acrescenta exatamente os elementos restantes. Cada elemento participa uma única vez do resultado. O caso-base fornece a soma correta do segmento unitário.

Há uma chamada sobre n/2 elementos: a = 1 e b = 2. O laço realiza n/2 iterações de custo constante, resultando em f(n) = Θ(n).

$$T_3(n) = T_3(n/2) + \Theta(n);\ T_3(1) = \Theta(1).$$

Tamanho	Parcela local para v = [1, ..., 8]	Soma local
8	5 + 6 + 7 + 8	26
4	3 + 4	7
2	2	2
1	Caso-base: 1	1

A soma final é 26 + 7 + 2 + 1 = 36 e o número de leituras é 4 + 2 + 1 + 1 = 8. O problema também admite varredura iterativa em Θ(n); a análise refere-se à implementação recursiva especificada.

ANÁLISE TEÓRICA

4.3 Problema 3 · enquadramento e prova

4.3.4 Aplicação do Caso 3

$a = 1$; $b = 2$; $p = \log_2 1 = 0$; $n^p = 1$.
No modelo de leituras: $f_Q(n) = n/2$.

O custo local é polinomialmente maior que n^p. Com $\varepsilon = 1$, $f_Q(n) = \Omega(n^{p+\varepsilon})$. A condição de regularidade, para tamanhos suficientemente grandes, é verificada por:

$a\, f_Q(n/b) = f_Q(n/2) = n/4$
$= (1/2) f_Q(n)$.

A escolha $c = 1/2 < 1$ satisfaz a regularidade. O Caso 3 estabelece $Q_3(n) = \Theta(n)$. A equivalência entre leituras e custo total, demonstrada a seguir, estende a classificação a $T_3(n)$.

4.3.5 Demonstração por indução

$Q_3(n)$ conta uma unidade por leitura no caso-base ou no laço. Assim, $Q_3(1) = 1$ e $Q_3(n) = Q_3(n/2) + n/2$. Afirma-se que $Q_3(n) = n$ para $n = 2^k$.

Base: para $k = 0$ e $n = 1$, a única leitura fornece $Q_3(1) = 1$.

Hipótese de indução: $Q_3(n/2) = n/2$.

Passo indutivo:

$$Q_3(n) = Q_3(n/2) + n/2 = n/2 + n/2 = n.$$

A igualdade vale para todas as potências de 2. Cada leitura e adição têm custo constante. O controle das chamadas acrescenta $O(\log n)$, limitado por $O(n)$, enquanto a execução das n leituras fornece $\Omega(n)$. Portanto, $T_3(n) = \Theta(Q_3(n)) = \Theta(n)$.

Os laços processam $n/2$, $n/4$, $n/8$, ... elementos; sua soma geométrica é linear. O custo não se repete com ordem n em todos os níveis. A memória auxiliar é $\Theta(\log n)$.

RESULTADOS EXPERIMENTAIS

5.1 Problema 1 · determinação do máximo

Estatísticas calculadas sobre sete médias de lote por tamanho. Q(n) é a contagem instrumentada; seus valores coincidiram integralmente com as expressões demonstradas. Tempos em µs.

n	Q observado	Mediana (µs)	Q1 a Q3 (µs)	c · g(n) (µs)
64	85	0,151490	0,149395 a 0,153222	0,153895
256	341	0,680714	0,645472 a 0,729911	0,615582
1.024	1.365	2,698525	2,580426 a 2,726288	2,462328
4.096	5.461	9,852143	9,781318 a 10,149947	9,849311
16.384	21.845	38,448528	37,710132 a 39,065071	39,397244
65.536	87.381	149,833791	147,075172 a 159,716429	157,588977
262.144	349.525	630,355906	621,238125 a 666,551047	630,355906
1.048.576	1.398.101	2.566,083000	2.507,862562 a 2.784,010438	2.521,423625

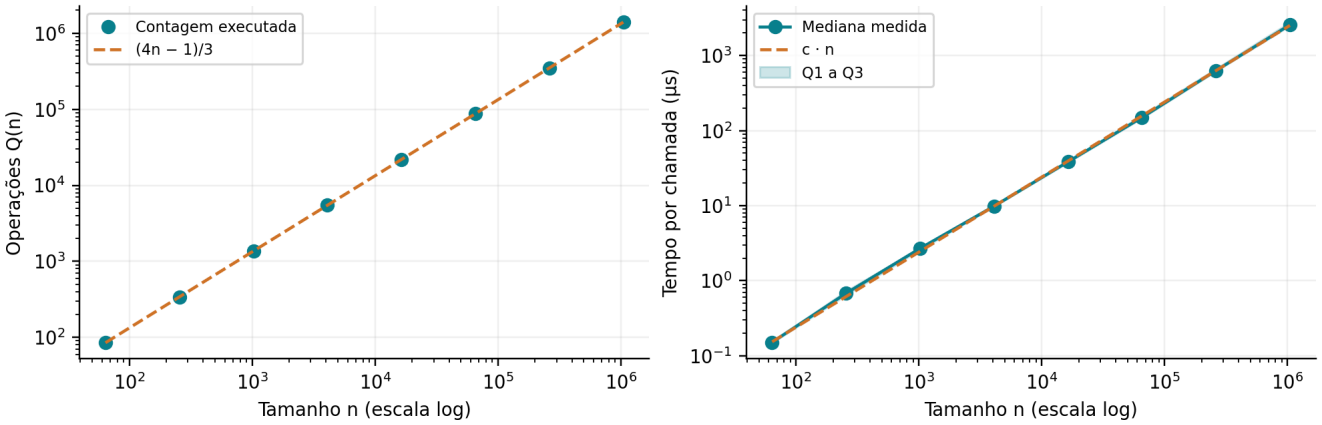


Figura 1. Contagem executada e expressão exata (esquerda); mediana temporal e curva calibrada (direita), com faixa Q1-Q3. Fonte: dados experimentais de 29/10/2026.

Na passagem de 262.144 para 1.048.576 elementos, a entrada é multiplicada por 4 e a mediana temporal aumenta por um fator de 4,071, próximo do fator linear 4. A contagem é exatamente (4n−1)/3 em todos os tamanhos. O desvio relativo entre a mediana e a curva calibrada varia de aproximadamente −4,9% a +10,6% no conjunto de pontos.

A tendência dos tempos é compatível com Θ(n). Variações de frequência, preempção e efeitos de memória constituem explicações possíveis para desvios, sem identificação causal nesta campanha. As contagens são exatas; a calibração temporal é descritiva.

RESULTADOS EXPERIMENTAIS

5.2 Problema 2 · busca binária

Estatísticas calculadas sobre sete médias de lote por tamanho. Q(n) é a contagem instrumentada; seus valores coincidiram integralmente com as expressões demonstradas. Tempos em µs.

n	Q observado	Mediana (µs)	Q1 a Q3 (µs)	c · g(n) (µs)
16	5	0,007159	0,007105 a 0,007406	0,006345
128	8	0,009888	0,009583 a 0,010503	0,010152
1.024	11	0,014151	0,014062 a 0,014983	0,013959
8.192	14	0,018895	0,018492 a 0,019428	0,017766
65.536	17	0,022648	0,022220 a 0,023523	0,021573
524.288	20	0,025380	0,025265 a 0,025902	0,025380
4.194.304	23	0,028202	0,027536 a 0,029076	0,029187
33.554.432	26	0,034247	0,033945 a 0,037693	0,032994

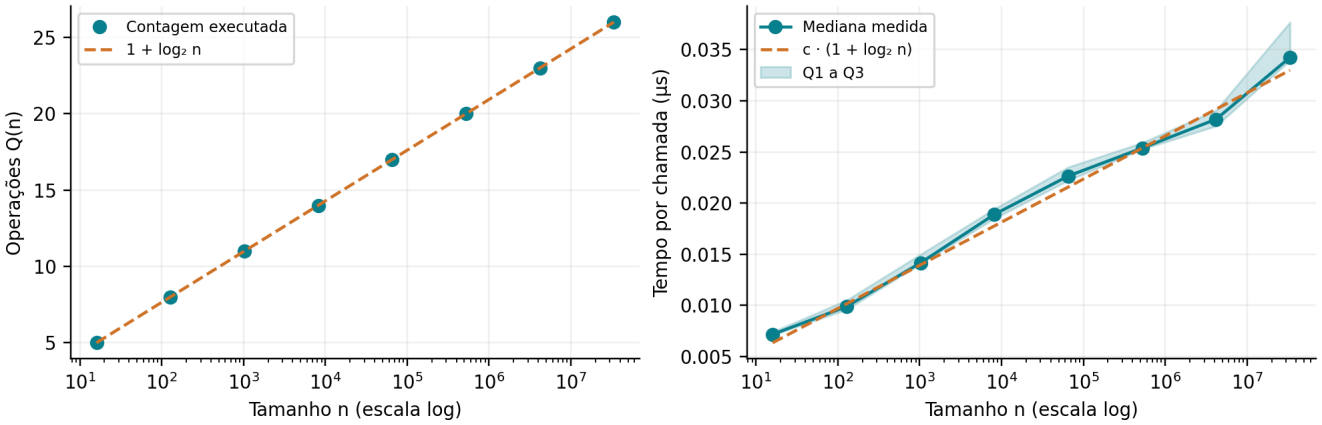


Figura 2. Contagem executada e expressão exata (esquerda); mediana temporal e curva calibrada (direita), com faixa Q1-Q3. Fonte: dados experimentais de 29/10/2026.

Entre tamanhos consecutivos, n é multiplicado por 8 e a contagem cresce três chamadas, conforme $\log_2(8n) = \log_2 n + 3$. Na última passagem, as contagens aumentam de 23 para 26 (fator 1,130), enquanto a mediana cresce 1,214 vezes. O desvio relativo em relação à curva calibrada varia de aproximadamente −3,4% a +12,8%. Os intervalos temporais são particularmente sensíveis a constantes do coletor.

Os gráficos adotam eixo horizontal logarítmico e vertical linear. A tendência é compatível com Θ(log n). A repetição do mesmo alvo favorece caches e predição de desvios; não permite generalizar os tempos para consultas com alvos aleatórios.

RESULTADOS EXPERIMENTAIS

5.3 Problema 3 · soma de elementos

Estatísticas calculadas sobre sete médias de lote por tamanho. Q(n) é a contagem instrumentada; seus valores coincidiram integralmente com as expressões demonstradas. Tempos em µs.

n	Q observado	Mediana (µs)	Q1 a Q3 (µs)	c · g(n) (µs)
64	64	0,029012	0,028413 a 0,031136	0,023778
256	256	0,091712	0,091020 a 0,092522	0,095112
1.024	1.024	0,342470	0,337008 a 0,349601	0,380447
4.096	4.096	1,697158	1,479727 a 1,731584	1,521788
16.384	16.384	5,813507	5,788416 a 5,954339	6,087153
65.536	65.536	24,348614	23,944878 a 25,154675	24,348614
262.144	262.144	94,833761	93,510022 a 96,578207	97,394454
1.048.576	1.048.576	417,811927	407,780500 a 451,386727	389,577816

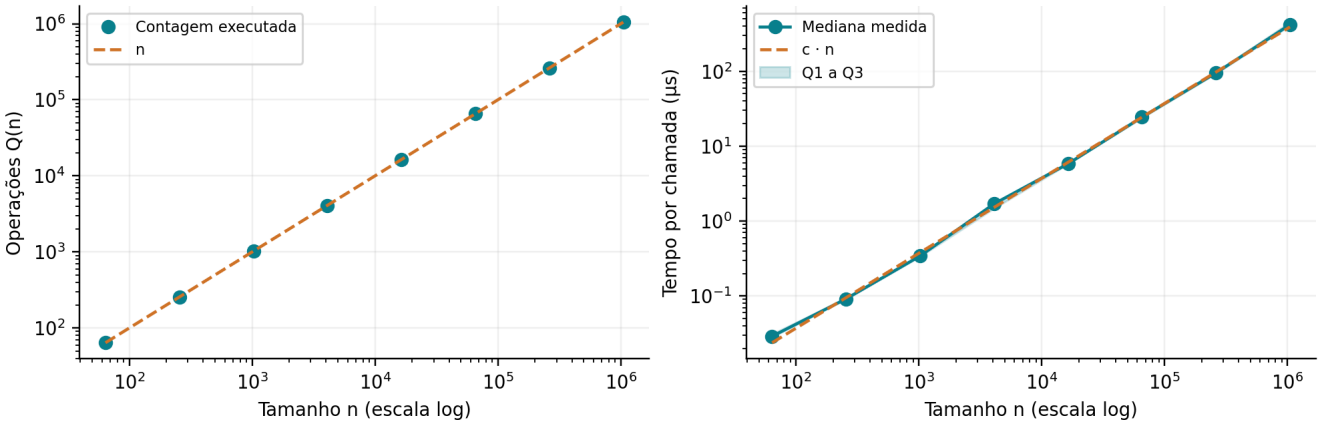


Figura 3. Contagem executada e expressão exata (esquerda); mediana temporal e curva calibrada (direita), com faixa Q1-Q3. Fonte: dados experimentais de 29/10/2026.

Na última passagem, a entrada é multiplicada por 4 e a mediana temporal aumenta 4,406 vezes. A contagem de leituras cresce exatamente por um fator de 4. Em relação à curva calibrada, os desvios temporais variam de aproximadamente −10,0% a +22,0%. O maior desvio ocorre em uma entrada de pequeno tamanho, na qual custos fixos possuem maior influência relativa.

A tendência dos tempos é compatível com Θ(n). Variações de frequência, preempção e efeitos de memória constituem explicações possíveis para desvios, sem identificação causal nesta campanha. As contagens são exatas; a calibração temporal é descritiva.

SÍNTESE

6. Conclusão

As recorrências dos três algoritmos satisfazem os casos estabelecidos do Teorema Mestre. As demonstrações por indução produziram contagens exatas e limites assintóticos coincidentes com os resultados do teorema. Todas as 24 contagens instrumentadas confirmaram essas expressões.

Algoritmo	Contagem exata	Complexidade
Máximo em quatro partes	$Q_1(n) = (4n - 1)/3$	Caso 1 · $\Theta(n)$
Busca binária	$Q_2(n) = \log_2 n + 1$	Caso 2 · $\Theta(\log n)$
Soma por recursão e laço	$Q_3(n) = n$	Caso 3 · $\Theta(n)$

6.1 Correspondência entre teoria e experimento

Os tempos apresentaram crescimento compatível com as classes linear e logarítmica previstas. Os fatores medidos nas últimas transições foram 4,071, 1,214 e 4,406, frente aos fatores de referência 4,000, 1,130 e 4,000. Essas diferenças não alteram as contagens e refletem a distinção entre custo abstrato e duração observada em ambiente compartilhado.

O maior intervalo interquartil relativo à mediana foi 12,4% no Problema 1, 10,9% no Problema 2 e 14,8% no Problema 3. As amostras descrevem dispersão entre lotes de uma campanha, sem estimar variabilidade entre máquinas ou campanhas independentes.

6.2 Implicações da modelagem

A classificação depende da relação entre f(n) e $n^{\log_b a}$. O máximo e a soma apresentam $\Theta(n)$, embora pertençam aos Casos 1 e 3. Na busca, alternativas mutuamente exclusivas resultam em $a = 1$. Na soma, a existência de um laço linear determina o custo local, sem acrescentar uma segunda chamada recursiva.

6.3 Limitações e continuidade

Os resultados são restritos a potências exatas, uma entrada determinística por tamanho e um ambiente de execução. Os tipos inteiros impõem limites finitos à implementação. Extensões incluem campanhas independentes em ambiente dedicado, alvos variados na busca e comparação com soluções iterativas, preservando a distinção entre algoritmo, modelo de custo e sistema de medição.

FONTES E REPRODUÇÃO

Referências e documentação dos arquivos

[1] ABEL, Z.; CHAPMAN, B.; DEMAINE, E. **Lecture 07: Recurrences.** MIT, Mathematics for Computer Science, Spring 2024. Seção 4, pp. 5-6. MIT OpenCourseWare. Acesso em: 29/10/2026.

[2] LINUX MAN-PAGES PROJECT. **clock_gettime(2), clock_getres(2).** Manual de chamadas do Linux. Acesso em: 29/10/2026.

Compilação e execução

O arquivo teorema_mestre.c contém os algoritmos e o coletor. A compilação requer GCC com suporte a noipa e um sistema POSIX. Os comandos a seguir geram executáveis de tempo e contagem e registram uma nova campanha em arquivo distinto.

```
gcc -std=c11 -O2 -Wall -Wextra -fno-optimize-sibling-calls \
    teorema_mestre.c -o tempo
gcc -std=c11 -O2 -Wall -Wextra -fno-optimize-sibling-calls \
    -DINSTRUMENT teorema_mestre.c -o contagem

./tempo 1 4 7 nova_execucao.csv
./contagem 1 4 1 nova_execucao.csv
./tempo 2 8 7 nova_execucao.csv
./contagem 2 8 1 nova_execucao.csv
./tempo 3 8 7 nova_execucao.csv
./contagem 3 8 1 nova_execucao.csv
```

Argumentos: problema, expoente k, amostras e CSV. Os exemplos usam n = 256: 4^4 no P1 e 2^8 nos P2/P3. Limites: $0 \leq k \leq 10$ no P1, $0 \leq k \leq 25$ no P2 e $0 \leq k \leq 20$ no P3. O modo de contagem executa uma verificação, independentemente do número de amostras informado.

Formato do CSV e componentes de reprodução

resultados_experimentos.csv contém os 192 registros originais, com delimitador vírgula e ponto decimal. As colunas são problema, n, expoente, modo, amostra, lote, segundos, operacoes, resultado e valido. segundos representa a média por chamada do lote. Campos não aplicáveis permanecem vazios; valido = 1 indica verificação aprovada.

O PDF incorpora o código, o CSV e os arquivos coletar.py, analisar.py, resumo.csv, ambiente.json, metricas.json, execucao.log e testar.c. A campanha completa é reproduzida por coletar.py e analisar.py em uma pasta de trabalho independente. O coletor usa afinidade do Linux e sobrescreve o CSV dessa pasta.

Repositório do trabalho

https://github.com/kaduolliver/Teorema-Mestre

APÊNDICE A · IMPLEMENTAÇÃO

Código-fonte C · parte 1 de 3

teorema_mestre.c, linhas 1 a 52. A numeração é editorial e não integra o arquivo compilável.

```
001  /* Analise experimental de algoritmos recursivos - C11, GCC e POSIX.
002   * Uso: ./tempo problema expoente amostras dados.csv
003   * n = 4^expoente no P1; n = 2^expoente no P2 e no P3.
004   * Compilar com -DINSTRUMENT para registrar contagens, sem tempo.
005   */
006  #define _POSIX_C_SOURCE 200809L
007  #include <errno.h>
008  #include <inttypes.h>
009  #include <stdint.h>
010  #include <stdio.h>
011  #include <stdlib.h>
012  #include <time.h>
013
014  #define REC __attribute__((noinline, noipa))
015  static uint64_t operacoes;
016  #ifdef INSTRUMENT
017  #define CONTA() (++operacoes)
018  #else
019  #define CONTA() ((void)0)
020  #endif
021
022  /* P1: maior elemento. Quatro chamadas de tamanho n/4.
023   * Conta uma visita por chamada. Precondicao: n = 4^k. */
024  REC uint32_t maior4(const uint32_t *v, size_t n) {
025      CONTA();
026      if (n == 1) return v[0];
027      size_t m = n / 4;
028      uint32_t a = maior4(v, m), b = maior4(v + m, m);
029      uint32_t c = maior4(v + 2*m, m), d = maior4(v + 3*m, m);
030      uint32_t maior = a > b ? a : b;
031      if (c > maior) maior = c;
032      if (d > maior) maior = d;
033      return maior;
034  }
035
036  /* P2: busca binaria. Escolhe uma metade e desce ate n = 1.
037   * Vetor ordenado; n = 2^k. Retorna 1 se encontrou, senao 0.
038   * Esta variante inclui o meio na metade direita, sem subtrair 1.
039   * Conta uma visita por chamada. */
040  REC int busca(const uint32_t *v, size_t n, uint32_t alvo) {
041      CONTA();
042      if (n == 1) return v[0] == alvo;
043      size_t m = n / 2;
044      if (alvo < v[m]) return busca(v, m, alvo);
045      return busca(v + m, m, alvo);
046  }
047
048  /* P3: soma. A primeira metade e recursiva; a segunda usa um laco.
049   * Conta uma leitura de elemento por folha ou iteracao. n = 2^k. */
050  REC uint64_t soma(const uint32_t *v, size_t n) {
051      if (n == 1) { CONTA(); return v[0]; }
052      size_t m = n / 2;
```

APÊNDICE A · IMPLEMENTAÇÃO

Código-fonte C · parte 2 de 3

teorema_mestre.c, linhas 53 a 104. A numeração é editorial e não integra o arquivo compilável.

```
053      uint64_t total = soma(v, m);
054      for (size_t i = m; i < n; ++i) { total += v[i]; CONTA(); }
055      return total;
056  }
057
058  static void erro(const char *msg) {
059      fprintf(stderr, "%s\n", msg); exit(EXIT_FAILURE);
060  }
061  static int numero(const char *s, int min, int max) {
062      char *fim; errno = 0;
063      long x = strtol(s, &fim, 10);
064      if (errno || fim == s || *fim || x < min || x > max)
065          erro("Argumento numerico invalido");
066      return (int)x;
067  }
068  static size_t potencia(size_t b, int k) {
069      size_t n = 1;
070      while (k-- > 0) n *= b;
071      return n;
072  }
073  static uint64_t executar(int p, const uint32_t *v, size_t n) {
074      if (p == 1) return maior4(v, n);
075      if (p == 2) return (uint64_t)busca(v, n, (uint32_t)n);
076      return soma(v, n);
077  }
078  #ifndef INSTRUMENT
079  static double agora(void) {
080      struct timespec t;
081      if (clock_gettime(CLOCK_MONOTONIC, &t)) erro("Falha no relogio");
082      return (double)t.tv_sec + (double)t.tv_nsec * 1e-9;
083  }
084  /* Mede o lote inteiro com apenas duas leituras de relogio.
085   * Acumular e verificar a saida preserva os resultados do lote.
086   * noipa impede memoizacao/eliminacao das chamadas repetidas. */
087  static double lote(int p, const uint32_t *v, size_t n, size_t b,
088                     uint64_t *checagem) {
089      uint64_t total = 0;
090      double inicio = agora();
091      for (size_t r = 0; r < b; ++r) total += executar(p, v, n);
092      double dt = agora() - inicio;
093      *checagem = total;
094      return dt;
095  }
096  #endif
097
098  int main(int argc, char **argv) {
099      if (argc != 5) erro("Uso: programa problema expoente amostras arquivo.csv");
100      int p = numero(argv[1], 1, 3);
101      int k = numero(argv[2], 0, p == 1 ? 10 : (p == 2 ? 25 : 20));
102      int amostras = numero(argv[3], 1, 99);
103      size_t n = potencia(p == 1 ? 4 : 2, k);
104      uint32_t *v = malloc(n * sizeof(*v));
```

APÊNDICE A · IMPLEMENTAÇÃO

Código-fonte C · parte 3 de 3

teorema_mestre.c, linhas 105 a 149. A numeração é editorial e não integra o arquivo compilável.

```
105     if (!v) erro("Falha de memoria");
106     /* Dados deterministas e reproduziveis: 0,1,...,n-1. Alvo n ausente. */
107     for (size_t i = 0; i < n; ++i) v[i] = (uint32_t)i;
108     uint64_t esperado = p == 1 ? n-1 :
109                         (p == 2 ? 0 : (uint64_t)n*(n-1)/2);
110     FILE *f = fopen(argv[4], "a+");
111     if (!f || fseek(f, 0, SEEK_END)) erro("Falha ao abrir CSV");
112     if (ftell(f) == 0)
113         fprintf(f, "problema,n,expoente,modo,amostra,lote,segundos,"
114                    "operacoes,resultado,valido\n");
115     uint64_t resultado = executar(p, v, n);
116     if (resultado != esperado) erro("Resultado incorreto");
117 #ifdef INSTRUMENT
118     (void)amostras;
119     uint64_t formula = p == 1 ? (4*(uint64_t)n-1)/3 :
120                        (p == 2 ? (uint64_t)k+1 : n);
121     if (operacoes != formula) erro("Contagem incorreta");
122     fprintf(f, "%d,%zu,%d,contagem,0,1,,%" PRIu64 ",%" PRIu64 ",1\n",
123             p, n, k, operacoes, resultado);
124 #else
125     uint64_t checagem;
126     size_t b = 1, limite = 1048576;
127     double dt;
128     /* Aquece e aumenta o lote ate 5 ms, ou ate o limite. */
129     do {
130         dt = lote(p, v, n, b, &checagem);
131         if (dt >= .005 || b == limite) break;
132         b *= 2;
133     } while (1);
134     double estimado = dt > 0 ? .020*b/dt : (double)limite;
135     b = estimado >= limite ? limite : (size_t)estimado + 1;
136     for (int a = 1; a <= amostras; ++a) {
137         dt = lote(p, v, n, b, &checagem);
138         /* Os limites de entrada mantem a soma do lote em uint64_t. */
139         if (checagem != esperado*b) erro("Resultado do lote incorreto");
140         fprintf(f, "%d,%zu,%d,tempo,%d,%zu,%.12g,,%" PRIu64 ",1\n",
141                 p, n, k, a, b, dt/b, resultado);
142     }
143     (void)operacoes;
144 #endif
145     if (ferror(f) || fclose(f)) erro("Falha ao gravar CSV");
146     free(v);
147     printf("P%d n=%zu resultado=%" PRIu64 " OK\n", p, n, resultado);
148     return 0;
149 }
```

APÊNDICE B · AMOSTRAS TEMPORAIS

Dados experimentais por configuração

Durações médias por chamada em µs, apresentadas com seis casas decimais. A1-A7 identificam as sete amostras de lote. O CSV contém os valores originais em segundos e o tamanho dos lotes.

P	n	A1	A2	A3	A4	A5	A6	A7
1	64	0,154406	0,149446	0,144892	0,154619	0,152038	0,149345	0,151490
1	256	0,837093	0,661700	0,598674	0,728640	0,731182	0,680714	0,629244
1	1.024	2,712221	2,940776	2,740354	2,698525	2,681116	2,472861	2,479736
1	4.096	9,767913	10,254223	10,045671	9,500501	10,277605	9,852143	9,794722
1	16.384	37,482002	39,052034	37,890958	40,525189	37,529306	38,448528	39,078109
1	65.536	145,633142	145,969104	164,715239	304,410075	149,833791	148,181239	154,717619
1	262.144	692,249781	673,940406	630,355906	625,557437	616,918813	659,161687	612,774781
1	1.048.576	2.608,421750	2.986,727500	2.959,599125	2.493,187375	2.522,537750	2.566,083000	2.464,215125
2	16	0,006246	0,007159	0,007062	0,007443	0,007439	0,007148	0,007373
2	128	0,011334	0,009888	0,010763	0,009714	0,010244	0,009451	0,009361
2	1.024	0,014646	0,013978	0,013792	0,016865	0,015321	0,014151	0,014145
2	8.192	0,019803	0,021665	0,018895	0,017457	0,018851	0,019053	0,018132
2	65.536	0,023310	0,025277	0,021868	0,022047	0,022393	0,022648	0,023737
2	524.288	0,026397	0,025247	0,025282	0,025407	0,025380	0,023724	0,026527
2	4.194.304	0,029767	0,027427	0,028385	0,027610	0,027462	0,030055	0,028202
2	33.554.432	0,034247	0,033927	0,033773	0,034589	0,033962	0,044521	0,040797
3	64	0,028197	0,028393	0,028433	0,029012	0,030273	0,034142	0,031999
3	256	0,093179	0,090811	0,090687	0,091229	0,091866	0,091712	0,093503
3	1.024	0,877968	0,342470	0,347664	0,336843	0,335150	0,351537	0,337173
3	4.096	1,697158	1,735002	1,479016	1,480437	1,464323	1,728165	2,675078
3	16.384	5,813507	5,811831	5,938721	5,739167	5,765000	5,969956	6,016653
3	65.536	23,912024	23,977731	23,476058	24,348614	24,883755	25,425595	27,673892
3	262.144	91,902152	96,092587	94,307739	94,833761	92,712304	97,063826	100,548848
3	1.048.576	442,430364	398,513855	417,047145	417,811927	460,343091	492,654364	392,389564

Fonte: resultados_experimentos.csv, campanha de 29/10/2026. Total: 168 registros de tempo e 24 contagens. Todos os 192 registros apresentam valido = 1.